Assignment #1 Basic Image Processing

測量115 F64111134 施淳譯

1. Explain and report the codes in Part A

a. Read and plot an image with OpenCV

```
img=cv2.imread("Lenna.png")
plt.figure(figsize=(4,4))
plt.imshow(img)
plt.title("Lenna orig")
plt.show()
```

- Read the image using cv2imread('img_name'), image is in BGR format.
- Create figure with plt.figure(figsize=(width,height)).
- Display the image using plt.imshow(img_name) .
- Set the figure title with plt.title('title_name').
- Show the figure with plt.show() in Jupyter/Ipython.

b. Fix the image color from BGR to RGB

```
img=cv2.cvtColor(img,cv2.COLOR_BGR2RGB)
plt.figure(figsize=(4,4))
plt.imshow(img)
plt.title("Lenna RGB")
plt.show()

# some image info:
print(type(img))
print(img.shape)
```

- Convert image color from BGR to RGB using cv2.cvtColor(img_name, cv2.COLOR_BGR2RGB), so the image displays correctly in matplotlib.
- Use type(img) to show the image data type, usually numpy array.
- Use image.shape to get the image shape (height, width, channels).

c. Show the channels of a color image

```
# show the R, G, and B channels
fig,ax_arr=plt.subplots(1,3,figsize=(12,4)) # fig -> the entire figure, ax_arr -> the subplots
ax_arr[0].imshow(img[:,:,0],cmap='gray')
# ax_arr[0] is the first subplot ,img[:,:,0] is the red channel, cmap set the image to gray
ax_arr[0].set_title("Red-channel image")
ax_arr[1].imshow(img[:,:,1],cmap='gray') # img[:,:,1] is the green channel
ax_arr[1].set_title("Green-channel image")
ax_arr[2].imshow(img[:,:,2],cmap='gray') # img[:,:,2] is the blue channel
ax_arr[2].set_title("Blue-channel image")
plt.show()
```

```
ax_arr[1].imshow(img[:,:,0], cmap="gray", vmin=0, vmax=255) # using absolute pixel scale intensity, low contrast
```

- Use plt.subplots(rows, cols, figsize=(width, height)) to create a figure and an array of subplots.
- Each subplots can be accessed with ax_arr[index].
- The color image is a three-dimensional array with (height, width, channels). Use image[:,:,index] to extract a single channel. 0 is red. 1 is green. 2 is blue.

- [:,:,index] means get the whole two-dimensional array with index channel.
- ‘cmap’ maps the pixel values to colors.
- Without vmin/vmax, imshow() auto-scales the pixel values using the actual min and max of the image -> higher contrast.
- With vmin and vmax, pixel values are mapped to the fixed range [vmin, vmax]. This ensures consistent scaling across images, but can reduce contrast if the actual values don’t span the full range.

2. Implement the functions of image thresholding and image histogram by using OpenCV in Part C with the aids of Germini or ChatGPT

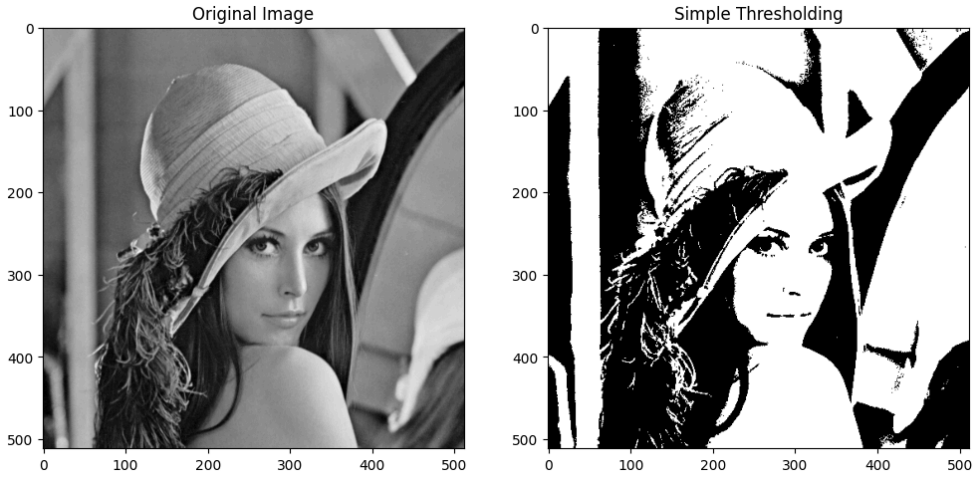
a. Image thresholding

```python
# Image thresholding
img = cv2.imread('Lenna.png', cv2.IMREAD_GRAYSCALE)

# Threshold
ret, thresh = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
# cv2.threshold(src, thresh, maxval, type)
# thresh: threshold value
# maxval: value to be given if pixel value is more than (sometimes less than)
# type: thresholding type, here we use simple binary thresholding

# Show images
fig, ax_arr = plt.subplots(1, 2, figsize=(12, 6))
ax_arr[0].imshow(img, cmap='gray')
ax_arr[0].set_title('Original Image')

ax_arr[1].imshow(thresh, cmap='gray')
ax_arr[1].set_title('Simple Thresholding')
plt.show()
```



b. Image histogram

```python
# Image histogram
img = cv2.imread('Lenna.png', cv2.IMREAD_GRAYSCALE)

# calculate histogram
hist = cv2.calcHist([img], [0], None, [256], [0,256])
# cv2.calcHist(images, channels, mask, histSize, ranges)
# use [img] because it expects a list of images
# [0] means gray image's channel
# histSize is the number of bins we want
# ranges is the range of pixel values

fig,ax_arr=plt.subplots(1, 2, figsize=(12,6))

# Show Original image
ax_arr[0].imshow(img, cmap='gray')
ax_arr[0].set_title('Original image')

# Plot Histogram image
ax_arr[1].plot(hist)
ax_arr[1].set_xlim([0, 255]) # set x-axis limits
ax_arr[1].set_xlabel('Bins')
ax_arr[1].set_ylabel('# of pixels')
ax_arr[1].set_title('Grayscale Histogram')
plt.show()
```

